

Device Login — Actions Backend Requisites

Holy Fork | Audit securite frontend | 23 juillet 2026

Contexte

Le device login permet a une tablette en salle de se connecter sans compte utilisateur classique. Le flow en 3 etapes :

1. Etape 1 — POST /api/auth/device-login/ : ID restaurant + PIN 6 chiffres → deviceToken

2. Etape 2 — GET /api/auth/restaurant-employees/ : liste des employes du restaurant

3. Etape 3 — POST /api/auth/quick-login/ : PIN employe 4 chiffres → JWT (access + refresh)

1. Problemes identifies

Sev.	Probleme	Impact
P0	Pas de contrainte unique sur le PIN restaurant	Deux restaurants peuvent avoir le meme PIN. Un attaquant qui brute-force un PIN peut tomber sur un restaurant qui n'est pas le sien.
P0	Pas de rate-limiting sur /api/auth/device-login/	Le PIN est 6 chiffres = 1 million de combinaisons. Sans throttle, un script peut tout tester en quelques minutes.
P1	Le PIN est renvoye en clair dans GET /api/restaurants/	Le champ pin est dans la reponse API. Visible dans le cache frontend (DevTools, TanStack Query).
P1	L'ID restaurant est un auto-increment previsible	Un attaquant peut iterer les IDs (1, 2, 3...) pour trouver des restaurants valides.
P2	Messages d'erreur distincts (enumeration)	Si le backend renvoie un message different pour "restaurant introuvable" vs "PIN incorrect", un attaquant peut identifier les IDs valides.

2. Actions requises

Action 1 — Contrainte unique sur le PIN P0

Ajouter unique=True sur le champ pin du modele Restaurant :

```
# models.py
class Restaurant(models.Model):
    pin = models.CharField(max_length=6, unique=True) # ← ajouter unique=True
```

```
# migration
python manage.py makemigrations
python manage.py migrate
```

Pourquoi : Avec ~1000 restaurants max et 1M de combinaisons possibles (6 chiffres), la probabilité de collision est négligeable. La contrainte unique garantit qu'un couple (ID, PIN) identifie toujours exactement 1 restaurant.

Si 2 restaurants ont déjà le même PIN : la migration échouera. Il faudra d'abord régénérer le PIN du doublon (`Restaurant.objects.filter(pin=X)`).

Action 2 — Rate-limiting sur device-login P0

Appliquer un throttle DRF sur l'endpoint `/api/auth/device-login/` :

```
# throttles.py
from rest_framework.throttling import AnonRateThrottle

class DeviceLoginThrottle(AnonRateThrottle):
    rate = '5/min' # 5 tentatives par minute par IP

    def get_cache_key(self, request, view):
        # Throttle par IP + restaurant_id pour plus de précision
        restaurant_id = request.data.get('restaurant_id', '')
        ident = f"{self.get_ident(request)}:{restaurant_id}"
        return self.cache_format % {'scope': self.scope, 'ident': ident}

# views.py
class DeviceLoginView(APIView):
    throttle_classes = [DeviceLoginThrottle]
    # ...
```

Pourquoi : 6 chiffres = 1M combinaisons. A 5 essais/min, il faudrait 138 jours pour tout tester. Sans throttle, c'est faisable en minutes avec un script.

Le frontend gère déjà le 429 — le code affiche "Trop de tentatives. Veuillez patienter." quand le backend renvoie HTTP 429.

Action 3 — Retirer le PIN des réponses API P1

Ne plus renvoyer le champ `pin` dans le serializer `RestaurantSerializer` utilisé par `GET /api/restaurants/` :

```
# serializers.py
class RestaurantSerializer(serializers.ModelSerializer):
    class Meta:
        model = Restaurant
```

```
exclude = ['pin'] # ← retirer pin de la reponse
# OU :
# fields = ['restaurant_id', 'name', 'address', ...] (sans 'pin')
```

Si le PIN doit être visible dans l'admin (pour le configurer), créer un serializer dédié :

```
class RestaurantAdminSerializer(serializers.ModelSerializer):
    """Serializer avec PIN – uniquement pour les endpoints admin."""
    class Meta:
        model = Restaurant
        fields = '__all__'
```

Pourquoi : Aujourd'hui le PIN est envoyé en clair dans la réponse `GET /api/restaurants/`. Il se retrouve dans le cache du navigateur (TanStack Query), visible dans les DevTools. Un employé qui ouvre les DevTools voit les PINs de tous ses restaurants.

Action 4 — Message d'erreur unique P2

La vue `DeviceLoginView` doit renvoyer le **même message** que l'ID soit invalide ou que le PIN soit faux :

```
# views.py – DeviceLoginView.post()
try:
    restaurant = Restaurant.objects.get(pk=restaurant_id)
except Restaurant.DoesNotExist:
    return Response({"detail": "Identifiants invalides"}, status=400)

if not hmac.compare_digest(restaurant.pin, pin_restaurant):
    return Response({"detail": "Identifiants invalides"}, status=400)
#             ^^^ meme message dans les 2 cas
```

Pourquoi : Si les messages sont différents ("Restaurant introuvable" vs "PIN incorrect"), un attaquant peut d'abord identifier les IDs valides, puis brute-forcer le PIN. Avec un message unique, il ne sait pas lequel est faux.

Bonus : `hmac.compare_digest()` fait une comparaison en temps constant, ce qui empêche les timing attacks (mesurer le temps de réponse pour deviner le PIN caractère par caractère).

3. Amélioration optionnelle — Code public

Remplacer l'exposition du PK séquentiel (`restaurant_id: 2`) par un code public non prédictible :

```
# models.py
import uuid

class Restaurant(models.Model):
    public_code = models.UUIDField(default=uuid.uuid4, unique=True, editable=False)
    # ...
```

```
# Le device login utilise public_code au lieu de restaurant_id
# POST /api/auth/device-login/
# { "public_code": "a1b2c3d4-...", "pin_restaurant": "951001" }
```

Pourquoi : Un auto-increment (1, 2, 3...) est devinable. Un UUID est imprevisible. L'attaquant ne peut plus iterer les IDs pour trouver des restaurants valides. C'est une couche de defense supplementaire, pas un remplacement du rate-limiting.

4. Resume des actions

Priorite	Action	Effort	Fichiers
P0	unique=True sur Restaurant.pin	5 min + migration	models.py
P0	Throttle 5/min sur device-login	15 min	throttles.py , views.py
P1	Retirer pin du serializer Restaurant	5 min	serializers.py
P2	Message d'erreur unique + hmac.compare_digest	10 min	views.py
Optionnel	UUID public au lieu du PK sequentiel	30 min	models.py , serializers.py , views.py

Impact frontend : Les actions P0, P1 et P2 ne cassent rien cote frontend. L'action optionnelle (UUID) necessite de modifier le champ restaurantId dans le formulaire device login — on s'en charge cote frontend une fois le backend pret.

Document genere dans le cadre de l'audit testing frontend Holy Fork. Contact : equipe frontend.